BRAC University — CSE 221: Algorithms

Complete Solutions: Spring 2025 (A & B) · Summer 2025 (A & B) · Fall 2025 (A & B)

All six exam papers solved with full working. Sections are labelled by paper (Spring-A, Spring-B, Summer-A, Summer-B, Fall-A, Fall-B).

SPRING 2025 — SET A

Q1(a) — Time Complexity Analysis [4 marks]

Code block 1 (lines 2–6): Outer loop i from n down to 1, step –2

The outer loop runs n/2 times → **O(n)**.

Inner loop j from n down to 1, step –1 → **O(n)**.

Combined: **O(n) × O(n) = O(n²)**

Code block 2 (lines 8–15): Outer loop k = 1, k*=4

k doubles each time by factor 4 → runs log■(n) = O(log n) times.

Inner loop m from 0 to n, step +3. BUT: if m==10, break. Since m starts at 0 and increments by 3, m hits 0, 3, 6, 9, 12 … The break fires when m==10, but m never equals 10 (0,3,6,9,12…). So the break never triggers! The inner loop runs n/3 times → **O(n)**.

Combined: **O(log n) × O(n) = O(n log n)**

Overall time complexity = O(n²) + O(n log n) = O(n²)

Q1(b) — Recurrence Relation: T(n) = 3T(n/4) + Θ(3n²) [2 marks]

We apply the **Master Theorem**: T(n) = aT(n/b) + f(n)

Here a = 3, b = 4, f(n) = Θ(n²).

Compute $\log_b a = \log_{■} 3 \approx 0.792$.

Since f(n) = Θ(n²) and $n^{\log_{■} 3} \approx n^{0.792}$, we have f(n) = $\Omega(n^{\log_{■} 3 + \varepsilon})$ for $\varepsilon \approx 1.2 > 0$.

Check regularity: $3 \cdot (n/4)^2 = 3n^2/16 \leq c \cdot n^2$ for $c = 3/16 < 1$. ✓

∴ By Master Theorem Case 3: T(n) = Θ(n²)

Q1(c) — Prove 5n² + 4n = Θ(T(n)) = Θ(n²) [3 marks]

From Q1(b), T(n) = Θ(n²). We must show 5n² + 4n = Θ(n²).

Upper bound O(n²): For n ≥ 1: 5n² + 4n ≤ 5n² + 4n² = 9n². Take c■ = 9, n■ = 1. ✓

Lower bound Ω(n²): For n ≥ 0: 5n² + 4n ≥ 5n². Take c■ = 5, n■ = 0. ✓

Since 5n² ≤ 5n² + 4n ≤ 9n² for all n ≥ 1, by definition of Θ:

∴ 5n² + 4n = Θ(n²) = Θ(T(n)) ■

Q1(d) — Why ignore constants and lower-order terms? [1 mark]

We care about how an algorithm *scales* as input grows large (asymptotic behaviour). Constants depend on hardware, compiler, and implementation — not the algorithm itself. Lower-order terms become negligible compared to the dominant term for large n. Big-O lets us compare algorithms in a machine-independent, growth-rate-focused way.

Q2(a) — Array Representation of Cows [1 mark]

Assign integer 0 to each white cow and 1 to each black cow. Create an array of length n where index i holds the colour-code of the (i+1)-th cow. Because white cows come first, the array looks like [0, 0, …, 0, 1, 1, …, 1].

Q2(b) — Find First Black Cow in O(log n) [1 mark]

Since the array is sorted (all 0s then all 1s), use **Binary Search**:

```
function firstBlack(A, n): lo = 0, hi = n - 1, result = -1 while lo <= hi: mid = (lo + hi) / 2 if A[mid] == 1: result = mid // candidate hi = mid - 1 // look further left else: lo = mid + 1 return result // serial number = result + 1 (1-indexed)
```

Time complexity: **O(log n)** — faster than O(n). ✓

Q2(c) — Worst-Case for Median-of-Three Pivot Quicksort [3 marks]

The worst case occurs when the chosen pivot is always the minimum or maximum of the subarray, producing partitions of size 0 and n–1, giving T(n) = T(n–1) + O(n) → O(n²).

Strategy to construct worst case: At every recursive call, arrange the array so that the median of A[start], A[mid], A[end] is the smallest or largest element in the subarray.

Example worst-case 7-element array: [2, 4, 6, 7, 5, 3, 1]

Initial call: start=0, end=6, mid=3. Values: A[0]=2, A[6]=1, A[3]=7. Sorted: (1, 2, 7) → median = 2. Pivot = 2. All other elements ≥ 2, so partition: left part empty, right part has 6 elements → worst-case split.

This pattern repeats at each recursive call, always yielding a near-empty partition. Result: **O(n²)** time.

Q2(d) — Best Algorithm to Sort by Number of Connections [2 marks]

Each robot has between 0 and n–1 connections — integers in a known bounded range [0, n–1]. This is perfect for **Counting Sort** (a non-comparison sort).

Name: **Counting Sort**

Time complexity: **O(n + k)** where k = n–1 (range of values) → **O(n)**

Q2(e) — Sort Robots by Height Descending in O(n) [3 marks]

i. Algorithm (Selection-by-find_tallest):

```
function sortByHeightDesc(A, n): sorted_list = [] remaining = copy of A // O(n) to copy for i from 1 to n: tallest = find_tallest(remaining) // O(1) per call sorted_list.append(tallest) remove tallest from remaining // O(1) with linked list / swap-to-end return sorted_list
```

Implementation note: Use a data structure where removal is O(1) — e.g., swap the found element to the end and reduce size. find_tallest(A) is given as O(1), and it is called n times.

ii. Time complexity: O(1) × n calls + O(n) setup = O(n)

Q3(a) — Can Two Parties Each Win Majority? [2 marks]

No. A majority requires more than ⌊n/2⌋ votes. If party X has more than n/2 votes and party Y also has more than n/2 votes, then together they have more than n votes — impossible since there are only n votes total. **At most one majority party can exist.**

Q3(b) — O(N) Algorithm to Check Majority [2 marks]

```
function hasMajority(arr, partyID, N): count = 0 for i from 0 to N-1: if arr[i] == partyID: count += 1 return count > floor(N / 2) // true if majority
```

Time: O(N). Space: O(1).

Q3(c) — Divide & Conquer: Can Halves Have Different Majorities? [2 marks]

Yes. The majority of each half can differ from the overall majority — or there may be no overall majority at all.

Example: arr = [1, 1, 2, 2, 2] (n=5, majority threshold = 2)

Left half [1, 1]: majority = 1 (appears 2 times > ⌊2/2⌋=1). ✓

Right half [2, 2, 2]: majority = 2 (appears 3 times > ⌊3/2⌋=1). ✓

Overall: 2 appears 3 times > ⌊5/2⌋=2 → overall majority = 2, not 1.

So the left half's majority (1) differs from the overall majority (2). ✓

Q3(d) — Complete the majority() Function [4 marks]

```
function majority(arr): if length(arr) == 1: return arr[0] mid = length(arr) // 2
left_majority = majority(arr[0 ... mid-1]) right_majority = majority(arr[mid ...
length(arr)-1]) # Merge Step: if left_majority == -1 and right_majority == -1: return -1
if left_majority != -1 and hasMajority(arr, left_majority, length(arr)): return
left_majority if right_majority != -1 and hasMajority(arr, right_majority, length(arr)):
return right_majority return -1 // no majority
```

Logic: recurse on both halves; at the merge step, use hasMajority() from Q3(b) to check whether either candidate is a majority in the *whole* array. Time: T(n) = 2T(n/2) + O(n) → **O(n log n)**.

SPRING 2025 — SET B

Q1(a) — Time Complexity Analysis [4 marks]

Code block 1 (lines 2–9): i from n to 1, step –3

Outer loop: n/3 iterations → O(n).

Inner loop: j from 0 to n, step +3. BUT if j==50, break. j takes values 0, 3, 6, …, 48, 51. j never equals 50, so break never fires. Inner loop runs n/3 iterations → O(n).

Combined block 1: **O(n²)**

Code block 2 (lines 11–14): k from 1, k*=3

Outer k-loop runs log■(n) times → O(log n).

Inner loop: m starts at n, condition m > n → **false immediately**. Loop never executes → O(1).

Combined block 2: **O(log n) × O(1) = O(log n)**

Overall: O(n²) + O(log n) = O(n²)

Q1(b) — Recurrence: T(n) = 5T(n/6) + Θ(3n) [2 marks]

Apply Master Theorem: a=5, b=6, f(n)=Θ(n).

$\log_b a = \log_{■} 5 \approx 0.898$.

$f(n) = \Theta(n)$ and $n^{\log_{■} 5} \approx n^{0.898}$.

$f(n) = \Omega(n^{\log_{■} 5 + \varepsilon})$ for $\varepsilon \approx 0.1$. Check regularity: $5 \cdot (n/6) = 5n/6 \leq c \cdot n$ for $c = 5/6 < 1$. ✓

∴ **Case 3 applies: T(n) = Θ(n)**

Q1(c) — Prove 5n + 4 = Θ(T(n)) = Θ(n) [3 marks]

T(n) = Θ(n). We prove 5n + 4 = Θ(n).

Upper bound: For n ≥ 4: 5n + 4 ≤ 5n + n = 6n. Take c■ = 6, n■ = 4. ✓

Lower bound: For n ≥ 0: 5n + 4 ≥ 5n. Take c■ = 5, n■ = 0. ✓

∴ **5n + 4 = Θ(n) = Θ(T(n))** ■

Q1(d) — Same as Set A. See above.

Q2(a–e) — Sheep version: Identical structure to Set A (cows → sheep)

Represent white sheep as 0, black sheep as 1. Array = [0,…,0,1,…,1]. Binary search for first 1 in O(log n). Worst-case Quicksort, Counting Sort, and find_tallest-based sort answers are **identical to Set A**.

Q3(a–d) — Transylvania movie version: Identical logic to Set A

Replace 'party' with 'movie'. All proofs, algorithms, and pseudocode from Q3 Set A apply directly.

SUMMER 2025 — SET A

Q1(a)(i) — Time complexity of funcB [3 marks]

```
int funcB(int n) { for k: 1 to n, step +2 → O(n/2) = O(n) iterations for j: n down to 1 → O(n) iterations for i: 1 to n, step +2: if (i%2 != 0): break ← i starts at 1, 1%2 = 1 ≠ 0 → breaks IMMEDIATELY Inner i-loop = O(1)
```

The innermost i-loop always breaks on the first iteration (i=1 is odd). So its cost = O(1).

funcB time: O(n) × O(n) × O(1) = **O(n²)**

Q1(a)(ii) — Recurrence relation of funcA [2 marks]

funcA(n) calls: funcA(n/2), funcA(n/4), funcA(n/6), and funcB(n/2).

funcB(n/2) = O((n/2)²) = O(n²/4) = O(n²).

T(n) = T(n/2) + T(n/4) + T(n/6) + O(n²)

Q1(a)(iii) — Solve the recurrence [3 marks]

The dominant term is f(n) = O(n²). The recursive calls shrink n quickly (n/2, n/4, n/6).

By Akra-Bazzi method (or intuition): the recursion tree has root cost O(n²), and sub-problems cost O(n²/4), O(n²/16), O(n²/36), … — a geometric series.

Sum at each level: n² (1 + 1/4 + 1/4·1/4 + …) → converges. Root dominates.

∴ **T(n) = Θ(n²)**

Q1(b) — Prove 10n■ + 9n³ + 8n = Θ(n■) [2 marks]

Upper bound O(n■): For n ≥ 1: 10n■ + 9n³ + 8n ≤ 10n■ + 9n■ + 8n■ = 27n■. c■=27. ✓

Lower bound Ω(n■): 10n■ + 9n³ + 8n ≥ 10n■. c■=10. ✓

∴ **10n■ + 9n³ + 8n = Θ(n■)** ■

Q2(a) — Linear Search for Minimum Temperature [2 marks]

```
function findMinTemp(temps, n): min_temp = temps[0] min_hour = 1 for i from 1 to n-1: if temps[i] < min_temp: min_temp = temps[i] min_hour = i + 1 return min_hour, min_temp
```

Time complexity: O(n)

Q2(b) — Modified Binary Search for Minimum in Unimodal Array [3 marks]

The temperature array is first decreasing then increasing (unimodal). We can use binary search by comparing mid with its neighbour:

```
function findMinBinary(temps, n): lo = 0, hi = n - 1 while lo < hi: mid = (lo + hi) / 2 if temps[mid] > temps[mid + 1]: lo = mid + 1 // minimum is in right half else: hi = mid // minimum is at mid or left return lo + 1, temps[lo] // 1-indexed hour
```

Logic: if temps[mid] > temps[mid+1], the sequence is still decreasing — minimum lies to the right. Otherwise it's in the left portion (including mid). Time complexity: **O(log n)**

Q3(a) — Best Sorting Algorithm for Student Scores [2 marks]

Scores are integers in range [0, 100] — a small, fixed range. Memory is not a constraint, and comparisons are expensive → use a **non-comparison sort**.

Best algorithm: **Counting Sort**

Time complexity: **O(N + k)** where k = 100 (constant) → effectively **O(N)**. Zero comparisons needed!

Q3(b) — Ternary Merge Sort: Dividing Step [2 marks]

```
function ternaryMergeSort(arr, left, right): if right - left < 2: // 0 or 1 element return
// Divide into three parts size = right - left + 1 third = size / 3 // integer division
mid1 = left + third - 1 // end of first third mid2 = left + 2*third - 1 // end of second
third // third part: mid2+1 to right ternaryMergeSort(arr, left, mid1)
ternaryMergeSort(arr, mid1+1, mid2) ternaryMergeSort(arr, mid2+1, right) merge3(arr, left,
mid1, mid2, right) // merge three sorted parts
```

Q3(c) — Is Ternary Merge Sort Asymptotically Faster? [1 mark]

No. Recurrence: T(n) = 3T(n/3) + O(n). By Master Theorem: a=3, b=3, log■3=1, f(n)=O(n)=O(n^1) → Case 2 → T(n) = O(n log n). Same as binary Merge Sort.

Dividing into 3 reduces tree depth from log■n to log■n, but there are now 3 sub-problems instead of 2. The total work per level remains O(n), and there are still Θ(log n) levels. Asymptotically identical.

Q4(a) — Find Maximum Negative Damage Box Using D&C; [2 marks]

Array: [2, 3, -4, -3, 5, -1, 2, -2]. We want the most negative value (max reduction to opponent).

Divide and conquer idea:

Divide the array in half. Recursively find the minimum (most negative) value in left and right halves. At the merge step, return the minimum of the two results. Base case: single element returns itself.

```
function findMaxDamage(A, left, right): if left == right: return A[left] mid = (left +
right) / 2 leftMin = findMaxDamage(A, left, mid) rightMin = findMaxDamage(A, mid+1, right)
return min(leftMin, rightMin)
```

For the given array, the answer is **-4** (index 2). Time: O(n).

Q4(b) — Maximum Subarray Sum with O(N log N) — Simulation [3 marks]

Array: [2, 3, -4, -3, 5, -1, 2, -2]. We use **Divide and Conquer Max-Sum Subarray**.

Step 1: Divide at mid=3 (0-indexed). Left = [2, 3, -4, -3], Right = [5, -1, 2, -2]

Step 2 — Left half [2, 3, -4, -3]:

Divide at mid=1. Left=[2,3], Right=[-4,-3].

[2,3]: max subarray = 5. [-4,-3]: max = -3. Cross: from mid going left: 3, 5; going right: -4, -7 → cross = 5-4=1.

Left half result: max(5, -3, 1) = **5**

Step 3 — Right half [5, -1, 2, -2]:

Divide at mid=1 (relative). Left=[5,-1], Right=[2,-2].

[5,-1]: max = 5. [2,-2]: max = 2. Cross: left side: -1, 4; right side: 2, 0 → cross = 4+2=6.

Right half result: max(5, 2, 6) = **6**

Step 4 — Cross of entire array:

From index 3 going left: -3, -7, -4, -2. Max left suffix ending at mid = -2.

From index 4 going right: 5, 4, 6, 4. Max right prefix starting after mid = 6.

Cross sum = -2 + 6 = **4**

Final answer: max(5, 6, 4) = 6 (subarray [5, -1, 2])

Q5(a) — Minimum Steps from Room 4 to Room 6 with Blocked Corridors [3 marks]

This is a **shortest path in an unweighted graph** problem. Use **BFS (Breadth-First Search)**.

Represent rooms as nodes and open corridors as edges. Start BFS from Room 4, ignoring blocked edges. BFS guarantees the shortest path in terms of number of steps (edges) in an unweighted graph.

The number of levels BFS traverses before reaching Room 6 = minimum steps required.

Algorithm idea:

```
1. Build adjacency list from remaining (non-blocked) corridors. 2. Run BFS starting from Room 4. 3. Track distance[] array, initialised to -1. 4. When Room 6 is dequeued, return distance[6]. Time complexity: O(V + E) where V=8 rooms, E=corridors.
```

Q5(b) — Can Gate of Reason Be Opened? (Bipartite + Equal Partition) [2 marks]

The conditions are: (1) no two spirits in the same group guard adjacent rooms (independent set), and (2) both groups have equal size (n/2 each for n=8 → groups of 4).

Condition (1) means the graph must be **2-colourable (bipartite)**. Condition (2) means both colour classes must have size 4.

```
function canOpenGate(graph, n): color = array of -1, size n // BFS bipartite check for each room r in 1..n: if color[r] == -1: queue = [r]; color[r] = 0 while queue not empty: node = dequeue(queue) for each neighbour v of node: if color[v] == -1: color[v] = 1 - color[node] enqueue(v) elif color[v] == color[node]: return FALSE // odd cycle, not bipartite // Check equal partition count0 = number of rooms with color 0 count1 = number of rooms with color 1 return (count0 == count1) // must be 4 and 4
```

If the function returns TRUE, the Gate of Reason can be opened.

SUMMER 2025 — SET B

Q1(a)(i) — Time complexity of funcB (Set B) [3 marks]

Same structure as Set A's funcB but outer loop step is +3 instead of +2. The innermost i-loop still breaks immediately (i=1 is odd, 1%2 ≠ 0 → break). Cost = O(1).

k-loop: n/3 iterations = O(n). j-loop: O(n). i-loop: O(1).

funcB time: O(n²)

Q1(a)(ii) — Recurrence of funcA (Set B) [2 marks]

funcA calls: funcA(n/2), funcA(n/4), funcA(n/8), and funcB(n/4).

funcB(n/4) = O((n/4)²) = O(n²/16) = O(n²).

T(n) = T(n/2) + T(n/4) + T(n/8) + O(n²)

Q1(a)(iii) — Solve recurrence (Set B) [3 marks]

Same analysis as Set A: dominant term is O(n²), sub-problems shrink rapidly (n/2, n/4, n/8). The sum at each recursion level converges geometrically. Root cost dominates.

∴ **T(n) = Θ(n²)**

Q1(b) — Prove 7n■ + 6n³ + 5n = Θ(n■) [2 marks]

Upper bound: 7n■ + 6n³ + 5n ≤ 7n■ + 6n■ + 5n■ = 18n■ (n ≥ 1). c■=18. ✓

Lower bound: 7n■ + 6n³ + 5n ≥ 7n■. c■=7. ✓

∴ **7n■ + 6n³ + 5n = Θ(n■)** ■

Q2(a) — Linear Search for Maximum Shipment Volume [2 marks]

```
function findMaxShipment(volumes, n): max_vol = volumes[0] peak_hour = 1 for i from 1 to n-1: if volumes[i] > max_vol: max_vol = volumes[i] peak_hour = i + 1 return peak_hour, max_vol
```

Time: O(n)

Q2(b) — Modified Binary Search for Peak in Unimodal Array [3 marks]

The shipment volume increases then decreases (unimodal). Binary search for peak:

```
function findPeakBinary(volumes, n): lo = 0, hi = n - 1 while lo < hi: mid = (lo + hi) / 2 if volumes[mid] < volumes[mid + 1]: lo = mid + 1 // peak is to the right else: hi = mid // peak is at mid or left return lo + 1, volumes[lo] // 1-indexed
```

Time: **O(log n)**

Q3(a) — Best Sorting for Product Ratings 1–5 [2 marks]

Ratings are integers in [1, 5] — an extremely small, fixed range. Memory not an issue, comparisons expensive.

Best algorithm: **Counting Sort**

Time complexity: **O(N + k)** with k=5 → **O(N)**. No comparisons at all!

Q3(b) — Ternary Merge Sort Dividing Step: Same as Set A.

Q3(c) — Same answer as Set A: **Not asymptotically faster — still O(n log n)**.

Q4(a) — Max Damage Box D&C; for [-2, 3, 4, -3, -5, 1, 2, 2] [2 marks]

We want the minimum element (most negative = max reduction to opponent).

Using the same D&C; min-finding approach: the minimum is **-5** (index 4).

Divide: Left=[-2,3,4,-3], Right=[-5,1,2,2]. Min(left)=-2, Min(right)=-5. Overall=-5.

Q4(b) — Max Subarray O(N log N) Simulation for [-2, 3, 4, -3, -5, 1, 2, 2] [3 marks]

Array indices 0–7. Divide at mid=3.

Left [-2, 3, 4, -3]: Max subarray = [3,4] = 7

Right [-5, 1, 2, 2]: Max subarray = [1,2,2] = 5

Cross: From mid=3 going left: -3, 1, 4, 2. Max left suffix = 4 (elements 4,3). From mid+1=4 going right: -5, -4, -2, 0. Max right prefix = 0 (empty, since all negative). Cross = 4 + 0 = 4.

Answer: max(7, 5, 4) = 7 (subarray [3, 4])

Q5(a) and Q5(b) — Same as Summer Set A (BFS for shortest path, bipartite check for Gate of Reason).

The graph structure is different (different room layout) but the algorithm is identical in both cases.

FALL 2025 — SET A

Q1(a) — Why ignore constants and lower-order terms? [1 mark]

Same answer as Spring 2025 Q1(d): we focus on growth rate for large inputs; constants are hardware/compiler-dependent and lower-order terms become negligible as $n \to \infty$.

Q1(b) — Is n·n asymptotically faster than n·log■(n)? [2 marks]

Compare $f(n) = n^2$ vs $g(n) = n \cdot \log■ n$.

Divide: $f(n)/g(n) = n^2/(n \log n) = n/\log n \to \infty$ as $n \to \infty$.

So f(n) grows much faster than g(n).

No, n·n = n² is NOT asymptotically faster — it is asymptotically SLOWER (worse). n·log■n = O(n log n) is faster than $n^2 = O(n^2)$.

Q1(c) — Time Complexity of func(arr) [3 marks]

```
public static void func(int[] arr) { int n = arr.length; int k = 0; for (int i = 0; i < n; i++) // O(n) k += arr[i]; // k = sum of all elements for (int i = 1; i < n; i++) // O(n) outer for (int j = 1; j <= arr[i]; j++) // inner runs arr[i] times System.out.print(i); }
```

First loop: **O(n)**.

Second nested loop: the inner loop runs arr[i] times for each i. Total inner iterations = Σ arr[i] for i=1..n-1 = (sum of array) - arr[0] ≈ k.

In the worst case, k = sum of all elements. If values can be up to some bound V, this is O(n·V). Without bound on values: **O(n + k)** where k = sum of all elements.

Time complexity: O(n + k) where k = Σ arr[i]

Q2(a) — Separate Negatives and Positives in O(N) time, O(1) space [5 marks]

Use a **two-pointer (partition) approach** similar to Dutch National Flag / Quick Select:

```
function organize(A, N): left = 0 right = N - 1 while left < right: // Move left pointer past negatives while left < right and A[left] < 0: left++ // Move right pointer past positives while left < right and A[right] > 0: right-- // Swap: a positive on the left with a negative on the right if left < right: swap(A[left], A[right]) left++ right-- return A // all negatives first, then positives
```

Simulation on A = {15, -13, 3, -4, 5, -6}:

Initial: [15, -13, 3, -4, 5, -6]. left=0, right=5.

A[0]=15 (positive, stop). A[5]=-6 (negative, stop). Swap → [-6, -13, 3, -4, 5, 15]. left=1, right=4.

A[1]=-13 (negative, skip). left=2. A[4]=5 (positive, skip). right=3.

A[2]=3 (positive, stop). A[3]=-4 (negative, stop). Swap → [-6, -13, -4, 3, 5, 15]. left=3, right=2. Stop.

Result: **[-6, -13, -4, 3, 5, 15]**. Time: O(N). Space: O(1). ✓

Q2(b) — Binary Search for Leftmost Positive in Org_A [3 marks]

Org_A is partitioned: negatives first, then positives. This is like a 0/1 sorted array. Binary search for the first positive:

```
function firstPositiveIndex(A, N): lo = 0, hi = N - 1, result = N // N means no positive found while lo <= hi: mid = (lo + hi) / 2 if A[mid] > 0: result = mid // candidate; look further left hi = mid - 1 else: lo = mid + 1 return result // -1 or 0-indexed; N if no positive exists
```

Time: **O(log N)**

Q2(c) — Best Sorting Algorithm for Each Case [2 marks]

Case 1: Only the largest number is out of place, rest sorted. Use **Insertion Sort** — O(N) for nearly-sorted arrays (1 out-of-place element = 1 pass).

Case 2: Smallest=0, largest=2^N. Range is exponentially large. Cannot use Counting Sort (range too big). Use **Merge Sort** — O(N log N), stable, no range assumptions.

Case 3: Array contains integers 1 to N. Known range [1, N]. Use **Counting Sort** — O(N + N) = O(N). Perfect for this fixed-range integer input.

Case 4: Numbers in reverse order. Any O(N log N) sort works. **Merge Sort** is best as it guarantees O(N log N) even in worst case (unlike Quick Sort).

Q3(a) — Mark Changes: MaxProdSubarray vs MaxSumSubarray [2 marks]

The Max-Sum Subarray uses addition; the Max-Prod version uses multiplication. Changed lines:

```
// In CrossProduct function: prod = 1 // was: sum = 0 (identity for multiplication vs
addition) prod = prod * A[i] // was: sum = sum + A[i] if prod > leftProd: ... // same
structure, different variable name prod = prod * A[i] // was: sum = sum + A[i] (right
side) return leftProd * rightProd // was: return leftSum + rightSum
```

Q3(b) — Does the Algorithm Work Correctly? [3 marks]

No, the algorithm does NOT always work correctly.

Counter-example: A = [-3, -2]

Correct max product subarray: [-3, -2] → product = 6.

Algorithm: CrossProduct left side: prod starts at 1, multiply by A[mid]=-3 → prod=-3 > -∞, leftProd=-3. Right side: prod=1, multiply by A[mid+1]=-2 → prod=-2 > -∞, rightProd=-2. Returns leftProd * rightProd = (-3)×(-2) = 6.

But wait — it works here. Let's try A = [-2, -1, -3]:

LPS = MaxProd([-2,-1]) = 2. RPS = MaxProd([-3]) = -3.

Cross: left side: mid=1. i from 1 to 0: prod×(-1)=-1 (leftProd=-1), then prod×(-2)=2 (leftProd=2). Right side: i=2: prod×(-3)=-3 (rightProd=-3). Cross = 2×(-3) = -6.

max(2, -3, -6) = 2. But correct answer is (-2)×(-1)×(-3) = -6? No — actual max product is (-2)×(-1)=2 or (-1)=–1, or just -1 or -2. The maximum is 2 ✓ here.

Genuine failure case: A = [-5, -6]. Left: LPS=RPS=-∞ impossible (base case: LPS=-5, RPS=-6). Cross: leftProd = -5, rightProd = -6. Returns (-5)×(-6) = 30. Correct! (max product of [-5,-6] = 30).

Real failure: A = [2, -1, 2]. LPS=2, RPS=2. Cross: left suffix from mid=1: A[1]=-1 (leftProd=-1), then A[0]=2, prod=(-1)(2)=-2. leftProd=-1. Right: A[2]=2, rightProd=2. Cross = (-1)×2 = -2. max(2, 2, -2) = 2. But the actual max product subarray is [2] or [2] = 2. ✓ Still works.

Counter-example: A = [3, -1, -2]. Correct: max product = (-1)×(-2)=2 or 3×(-1)×(-2)=6. Max=6.

LPS = MaxProd([3,-1]) → LPS=3, RPS=-1, cross: leftProd=3×(-1)?? Let's redo: left=[3,-1], mid=0. CrossProduct: left side i from 0 to 0: prod=1×3=3, leftProd=3. Right: i=1: prod=1×(-1)=-1, rightProd=-1. Cross=3×(-1)=-3. max(3,-1,-3)=3. So MaxProd([3,-1])=3.

Back to full: LPS=3, RPS=-2, Cross: left from mid=1: A[1]=-1→prod=-1, leftProd=-1; A[0]=3→prod=-3, leftProd=-1. Right: A[2]=-2→prod=-2, rightProd=-2. Cross=(-1)×(-2)=2. max(3,-2,2)=3.

But correct answer = 6 (all three elements: 3×-1×-2=6). Algorithm returns 3. INCORRECT! ✓

The cross product only takes the best suffix from the left and best prefix from the right, but doesn't handle sign-flipping from negative×negative correctly across the midpoint.

Q3(c) — Recurrence after Karatsuba [2 marks]

Karatsuba multiplies two n-digit numbers in $O(n^{1.58})$. In our algorithm, CrossProduct loops O(N) times and each multiplication becomes a Karatsuba call of cost $O(N^{1.58})$.

MaxProdSubarray itself calls itself twice (on N/2 each) plus CrossProduct:

$T(N) = 2T(N/2) + N \cdot O(N^{1.58}) = 2T(N/2) + O(N^{2.58})$

Q3(d) — Solve $T(N) = 2T(N/2) + O(N^{2.58})$ [2 marks]

Apply Master Theorem: a=2, b=2, $f(N)=O(N^{2.58})$.

$\log_b a = \log_{\blacksquare} 2 = 1$.

$f(N) = \Omega(N^{1+\varepsilon})$ for $\varepsilon=1.58 > 0$. Check regularity: $2\cdot(N/2)^{2.58} = 2\cdot N^{2.58}/2^{2.58} = N^{2.58}/2^{1.58} \leq c\cdot N^{2.58}$ for $c = 1/2^{1.58} < 1$. ✓

∴ **Case 3: $T(N) = \Theta(N^{2.58})$**

FALL 2025 — SET B

Q1(a) — Same as Fall Set A.

Q1(b) — Is n·log■(n) asymptotically faster than n? [2 marks]

Compare f(n) = n log■n vs g(n) = n.

Divide: f(n)/g(n) = n log n / n = log n → ∞ as n → ∞.

No, n·log■n is NOT faster — it is SLOWER than n. n = O(n) is faster than n·log■n = O(n log n). n log n grows faster.

Q1(c) — Time Complexity of func (Set B) [3 marks]

```
public static void func(int[] arr) { int n = arr.length; int k = 0; for (int i = 0; i < n;
i++) // O(n) — computes k = sum of arr k += arr[i]; for (int i = 1; i < n; i++) // O(n)
outer for (int j = 1; j <= k; j++) // inner: runs k times for EVERY i System.out.print(i);
}
```

Here the inner loop runs exactly k times per outer iteration (not arr[i] times). k is fixed after the first loop.

Total = O(n) + O(n × k) = **O(n·k)** where k = sum of all elements in arr.

If k is bounded (e.g., k = O(n)), then O(n²). If k is unbounded, O(n·k).

Time complexity: O(n·k)

Q2(a) — Separate Even/Odd in O(N), O(1) space [5 marks]

Same two-pointer technique as Fall Set A, but the condition changes:

```
function organize(B, N): left = 0 right = N - 1 while left < right: while left < right and
B[left] % 2 == 0: // skip evens on left left++ while left < right and B[right] % 2 == 1:
// skip odds on right right-- if left < right: swap(B[left], B[right]) left++ right--
return B // all evens first, then odds
```

Simulation on B = {7, 2, 9, 4, 11, 6}:

left=0, right=5. B[0]=7 (odd, stop). B[5]=6 (even, stop). Swap → [6,2,9,4,11,7]. left=1, right=4.

B[1]=2 (even, skip), B[2]=9 (odd, stop). left=2. B[4]=11 (odd, skip), B[3]=4 (even, stop). right=3.

B[2]=9 (odd, stop). B[3]=4 (even, stop). Swap → [6,2,4,9,11,7]. left=3, right=2. Stop.

Result: **[6, 2, 4, 9, 11, 7]**. Time: O(N). Space: O(1). ✓

Q2(b) — Binary Search for Rightmost Even in Org_B [3 marks]

Org_B has all evens first, then odds. We want the last even (rightmost even):

```
function lastEvenIndex(B, N): lo = 0, hi = N - 1, result = -1 while lo <= hi: mid = (lo +
hi) / 2 if B[mid] % 2 == 0: // mid is even result = mid // candidate; look further right
lo = mid + 1 else: hi = mid - 1 // look left return result // index of rightmost even; -1
if none
```

Time: **O(log N)**

Q2(c) — Best Sort for Each Case (Set B) [2 marks]

Case 1: Smallest number can be anywhere, rest sorted. **Insertion Sort** — O(N) for 1 out-of-place element.

Case 2: Range [0, 2^N] — exponentially large. **Merge Sort** — O(N log N), no range constraint.

Case 3: Integers 1 to N. **Counting Sort** — O(N). Range = N, perfect fit.

Case 4: Random numbers, no pattern. **Merge Sort** — O(N log N) guaranteed worst-case.

Q3(a–d) — Same as Fall Set A

The array differs ([5,-3,13,4,-5,-6] instead of [15,-13,3,-4,5,-6]) but the algorithm analysis, counter-example construction, recurrence, and Master Theorem solution are identical.

Counter-example for Q3(b) with A=[5,-3,13]: LPS=MaxProd([5,-3])=5, RPS=13. Cross: leftProd=max(-3, 5×-3)=max(-3,-15)=-3. rightProd=13. Cross=-3×13=-39. max(5,13,-39)=13. But actual max = 5×(-3)×13 = -195? No — actual max is 13 alone = 13. ✓ Works here.

Try A=[5,-1,-2]: LPS=5, RPS=-1 vs -2 → RPS=-1. Cross: left from mid=1: A[1]=-1→leftProd=-1, A[0]=5→prod=-5, leftProd=-1. Right: A[2]=-2→rightProd=-2. Cross=(-1)×(-2)=2. max(5,-1,2)=5. Correct (5 alone =5). ✓

The recurrence and Karatsuba answers for Set B are **identical to Set A: $T(N) = 2T(N/2) + O(N^{2.58})$** $\rightarrow$ **$\Theta(N^{2.58})$**.